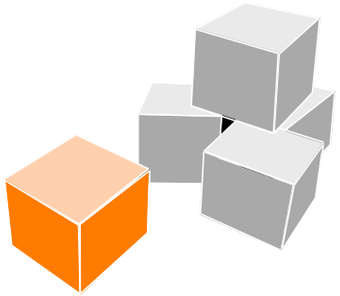
Concepts of Programming Languages

PARSING, CONDITIONALS, NAMES

Prof. Dr. Mira Mezini



Software Technology Group
TU Darmstadt | FB Informatik

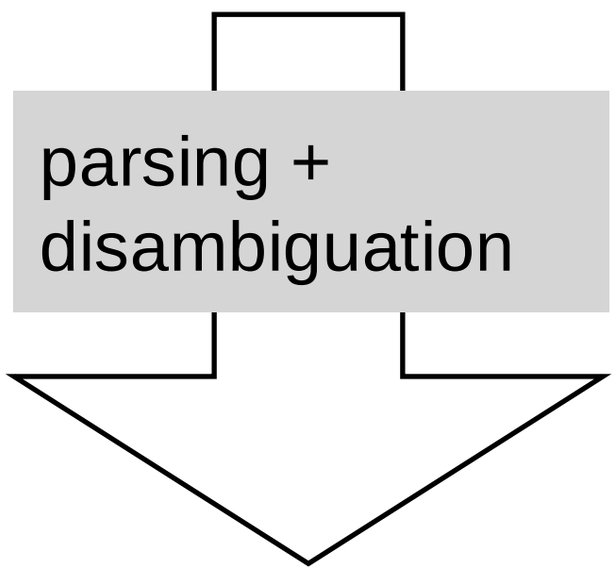
PARSING

How to go from concrete syntax to abstract syntax?

Parsing

Concrete syntax definition
— defines program *text*; recipe for parsing and disambiguation

Program in concrete syntax



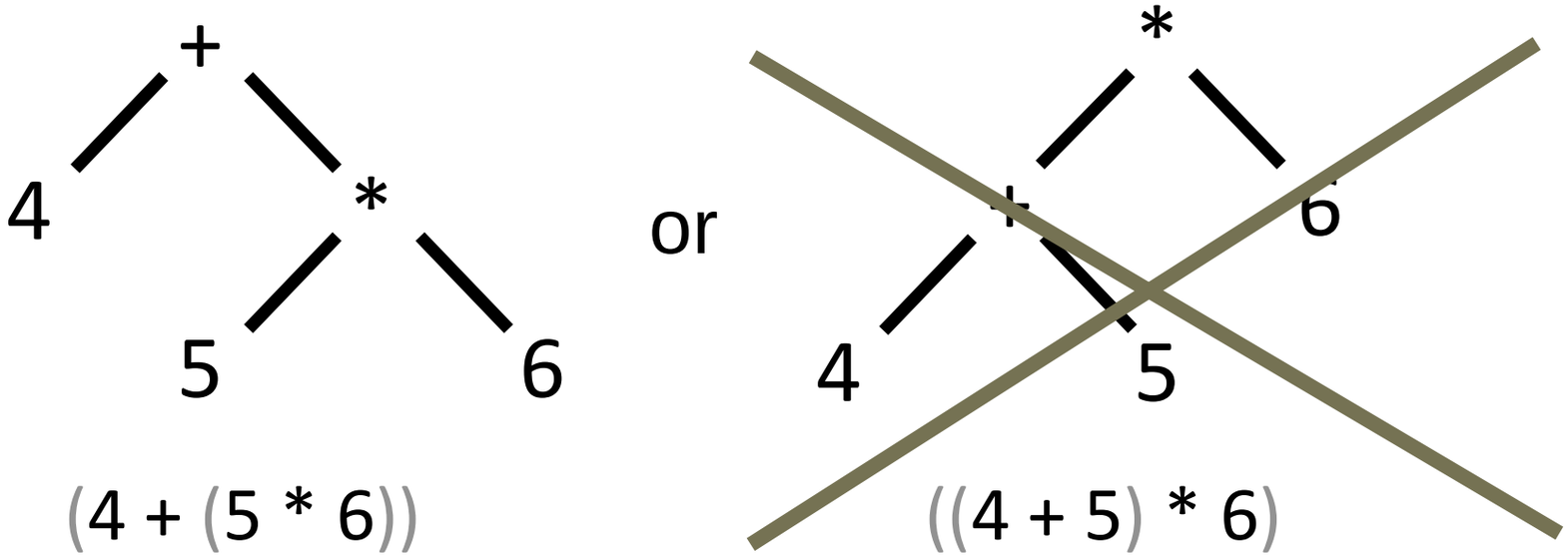
Abstract Syntax Tree

Abstract syntax representation
— defines program *structure*; no irrelevant details

In general, the process of converting the concrete syntax into ASTs is called parsing

Parsing & Disambiguation by Example

4 + 5 * 6



(4 + (5 * 6))

((4 + 5) * 6)

Parsing — A Subject of Study on its Own

Parser generators

SDF ANTLR yacc

Bison SableCC Happy

…

Parser combinators in

Haskell Scala Python

…

Parsing algorithms

GLR SGLR LALR(1)

LL(k) LR(1) Early

…

No parsing in the rest of this course

We avoid the topic of parsing altogether.

We rely on very simple concrete syntax
- Scala AST representation
- S-Expressions

S-expressions

4 + 5 * 6 $\Rightarrow$ (+ 4 (* 5 6))

An S-expression is either:

- A literal
  - Symbol, Number, String
- A list of s-expressions
  - In parenthesis, separated by spaces

```
enum SExp:
 case SList(list: List[SExp])
 case SSym(symbol: String)
 case SNum(num: Int)
 case SString(string: String)
```

S-expressions

Scala AST		S-Expr
Num(1)	⇒	1
Add(Num(1) Num(2))	⇒	(+ 1 2)
Add(Num(1) Add(Num(2) Num(3))	⇒	(+ 1 (+ 2 3))

Concrete Syntax as s-expressions

We define valid expressions using Backus-Naur-Form grammars

```
Exp ::= Int              (Num)
      | (+ Exp Exp)      (Plus)
      | (* Exp Exp)      (Mult)
```

```
enum Exp:
 case Num(n: Int)
 case Plus(l: Exp, r: Exp)
 case Mult(l: Exp, r: Exp)
```

S-Expression Parsing

```
(+ 23
  (* 5 6))
```

⇒

```
SList(List(
  SSym("+"),
  SNum(23),
  SList(List(
    SSym("*"),
    SNum(5),
    SNum(6)))))
```

⇒

```
Plus(
  Num(23),
  Mult(
    Num(5),
    Num(6)))
```

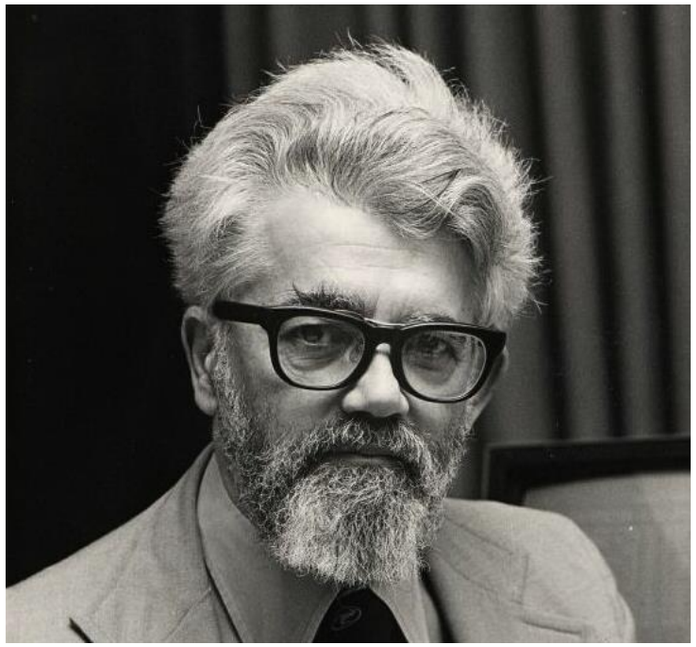
Concrete Syntax

S-Expression Syntax

Abstract Syntax

Lisp

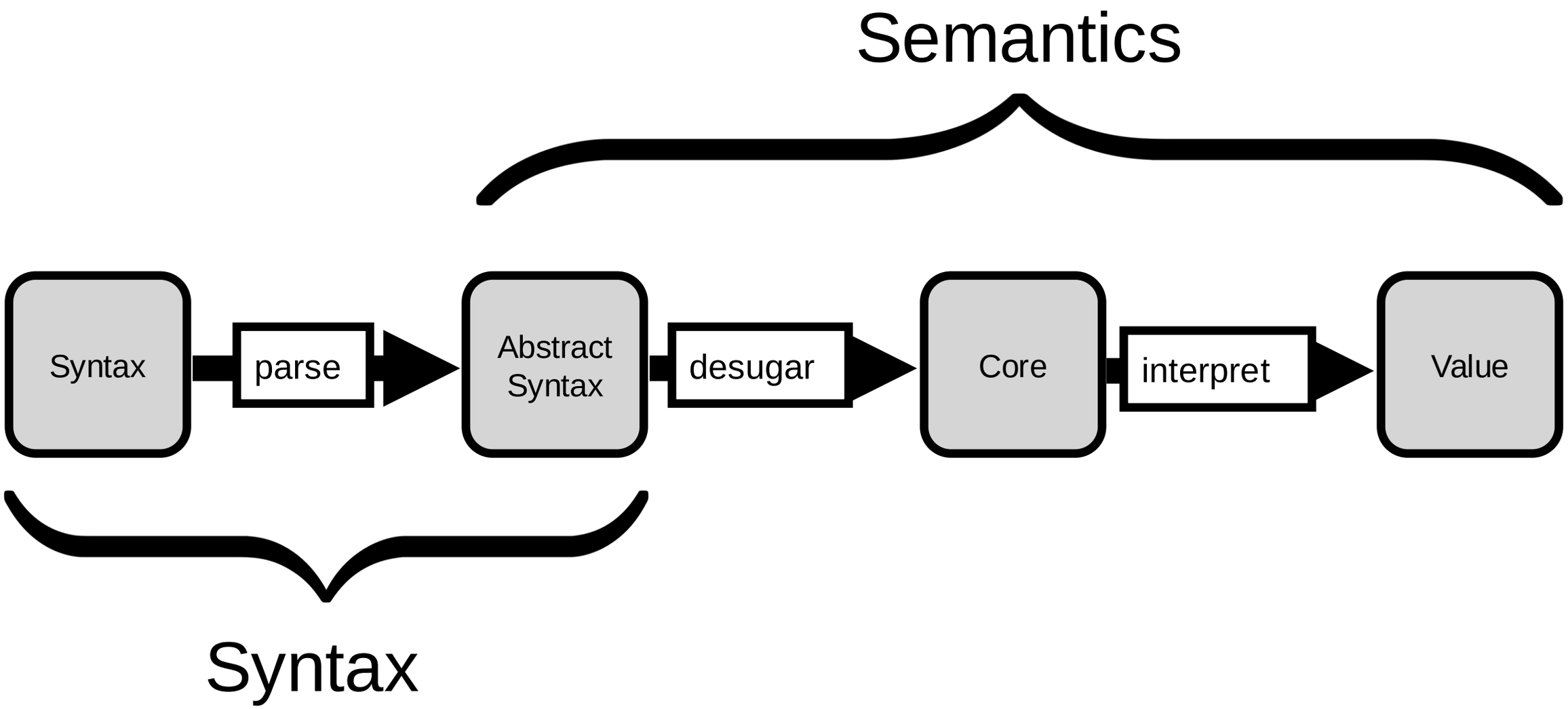
John McCarthy (Sep. 4, 1927 – Oct.24, 2011) was an American computer scientist and cognitive scientist. He coined the term "artificial intelligence" (AI), developed the *Lisp* programming language family, significantly influenced the design of the *ALGOL* programming language, popularized timesharing, and was very influential in the early development of AI. McCarthy received many accolades and honors, such as the *Turing Award* for his contributions to the topic of AI, the United States National Medal of Science, and the Kyoto Prize.



Around 1959, he invented so-called "garbage collection" methods to solve problems in Lisp. Based on the lambda calculus, Lisp soon became the programming language of choice for AI applications after its publication in 1960.

source: https://en.wikipedia.org/wiki/John_McCarthy_(computer_scientist)

Syntax versus Semantics



CONDITIONALS

There can be very complex conditional expressions in real languages.
For now we'll keep it simple and will only consider a simple if-then-else.

How Proceed to Add Conditionals

How would you proceed to add conditionals?

How Proceed to Add Conditionals

1. Extend the datatype representing expressions to include conditionals

2. Extend the interpreter to handle (the representation of) conditional expressions

3. Extend the parser if we have one to produce the new representations

1. Extending the Abstract Representation

```scala
sealed trait Exp

case class Num(n: Int) extends Exp

case class Add(lhs: Exp, rhs: Exp) extends Exp
case class Sub(lhs: Exp, rhs: Exp) extends Exp
case class Cond(c: Exp, t: Exp, e: Exp) extends Exp
```

2. Extending the Interpreter…

What is the meaning of the following?

`if (e) { e1 } else { e2 }`

```
def interp(expr: AE): Int = expr match {
  case Num(n)          => n
  case Add(lhs, rhs) => interp(lhs) + interp(rhs)
  case Sub(lhs, rhs) => interp(lhs) - interp(rhs)

  case Cond(c, t, e) => ... interp(c)
                        ... interp(t)
                        ... interp(e)
}
```

what's the result of **`interp(c)`**?

Even this simple construct results in several, mostly independent design decisions.

What are they?

The Design Space of Conditionals

1. What kind of terms can the test-phrase be?
2. What kind of program phrases (terms) are the branches? Expressions or statements?
3. If branches are expressions and evaluate to values, how do their values relate? Are they of the same type?

What examples do you know?

```
def interp(expr: AE): Int = expr match {
  case Num(n)          => n
  case Add(lhs, rhs) => interp(lhs) + interp(rhs)
  case Sub(lhs, rhs) => interp(lhs) - interp(rhs)

  case Cond(c, t, e) => ... interp(c)
                        ... interp(t)
                        ... interp(e)
}
```

The Test-Expression

In some languages the test expression must evaluate to Boolean values (one representing truth and the other falsehood).

Other languages use (several) truthy/falsy values

Value	JS	Perl	PHP	Python	Ruby
`0`	falsy	falsy	falsy	falsy	truthy
`""`	falsy	falsy	falsy	falsy	truthy
`NaN`	falsy	truthy	truthy	truthy	truthy
nil/null/None/undef	falsy	falsy	falsy	falsy	falsy
`"0"`	truthy	falsy	falsy	truthy	truthy
`-1`	truthy	truthy	truthy	truthy	truthy
`[]`	truthy	truthy	falsy	falsy	truthy
empty map/object	truthy	falsy	falsy	falsy	truthy

2. Extending the Interpreter

What about this?

```
def interp(expr: AE): Value = expr match {
  case Num(n)          => n
  case Add(lhs, rhs) => interp(lhs) + interp(rhs)
  case Sub(lhs, rhs) => interp(lhs) - interp(rhs)

  case Cond(c, t, e) => if (interp(c) == 0)
                                  interp(t) else interp(e)
}
```

So, we use 0 to represent true and anything else is considered false …

We have implemented a kind of **`if0`** construct, which is fine given that our language has only Ints.

2. Extending the Interpreter

So, we have numbers and conditionals …

But one could say that all we've done is (mostly) deferring to Scala to handle these.

Do you (dis)agree?

Let’s Add Proper Boolean Expressions

What do we need to do?

Let's Add Proper Boolean Expressions

Well, we have a recipe …

1. Extend the datatype representing expressions to include ~~conditional~~ boolean expressions
2. Extend the interpreter to handle (the representation of) ~~conditional~~ boolean expressions
3. Extend the parser if we have one to produce the new representations

Do you think applying the recipe blindly will work?
If no, why not?

Extending the Program Representation

Adding Boolean terms is easy …

```
sealed trait Exp

case class Num(n: Int) extends Exp

case class Bool(b: Boolean) extends Exp

case class Add(lhs: Exp, rhs: Exp) extends Exp
case class Sub(lhs: Exp, rhs: Exp) extends Exp
case class Cond(c: Exp, t: Exp, e: Exp) extends Exp
```

Num and **Bool** constructors are capturing **syntactic *constants*** in the program. This is *abstract syntax*: we are (abstractly) representing the *program that the user wrote*, **not the result of its evaluation**.

They do not represent compound expressions that will evaluate to numbers or Booleans. What an expression will evaluate to, **for now**, can only be determined by running it.

Are there other ways?

Extending the Interpreter …

… Like This?

```
def interp(expr: Exp): Int = expr match {
  case Num(n)          => n
  case Bool(b)         => b
  case Add(lhs, rhs) => ...
  case Sub(lhs, rhs) => ...
  case Cond(c, t, e) => interp(c) match
      case true  => interp(t)
      case false => interp(e)
}
```

Extending the Interpreter …

… Like This?

Found: Bool
Required: Int

Found: Int
Required: Bool

```
def interp(expr: Exp): Int = expr match {
  case Num(n)          => n
  case Bool(b)         => b
  case Add(lhs, rhs) => ...
  case Sub(lhs, rhs) => ...
  case Cond(c, t, e) => interp(c) match
      case true  => interp(t)
      case false => interp(e)
}
```

Design Decision: The Range of Values

- We aren't interested only in arithmetic expressions; we want to build full-fledged programming languages.
- They have a wide range of values (i.e., answers): numbers, Boolean, strings, images, functions, and more…

Thinking about the range of supported values is another aspect of language design.

Implementing the Decision About Values

We first define a datatype that reflects the different kinds of values that an interpreter can produce. Convention: call the return type constructors ...V to distinguish from the inputs of the interpreter

```
sealed trait Val
case class NumV(n: Int) extends Val
case class BoolV(b: Boolean) extends Val
```

```
def interp(expr: Exp): Val = expr match {
  case Num(n)          => NumV(n)
  case Bool(b)         => BoolV(b)
  case Add(lhs, rhs) => ...
  case Sub(lhs, rhs) => ...  ???
  case Cond(c, t, e) => ...
}
```

Need to map between values in our language and values of Scala

Implementing the Decision About Values

```
def interp(expr: Exp): Val = expr match {
  case Num(n)          => NumV(n)
  case Bool(b)         => BoolV(b)

  case Add(lhs, rhs) =>
       NumV(unboxNum(interp(lhs)) + unboxNum(interp(rhs)))
  case Sub(lhs, rhs) => ...

  case Cond(c, t, e) =>
        if unboxBool(interp(c)) interp(t)
        else interp(e)
}
```

```
def unboxNum(v: Val): Int = v match {
  case NumV(n) => n
  case _ => sys.error(s"expected Val.NumV but got $this")
}

def unboxBool(v: Val): Boolean = v match {
  case BoolV(b) => b
  case _ => sys.error(s"expected Val.BoolV but got $this")
}
```

We are make a decision here: We are explicitly saying in the interpreter code that we only allow to add numbers.

Encapsulating Design Decision Points

```
def interp(expr: Exp): Val = expr match {
  case Num(n)          => NumV(n)
  case Bool(b)         => BoolV(b)
  case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
  case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
  case Cond(c, t, e) => boolDec(interp(c)) match
      case true  => interp(t)
      case false => interp(e)
}
```

```
def addDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) + unboxNum(rhs))

def subDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) - unboxNum(rhs))

def boolDec(test: Val): Boolean = unboxBool(test)
```

Encapsulating Design Decision Points

```
 def interp(expr: Exp): Val = expr match {
  case Num(n)         => NumV(n)
  case Bool(b)        => BoolV(b)
  case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
  case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
  case Cond(c, t, e) => boolDec(interp(c)) match
      case true  => interp(t)
      case false => interp(e)
}
```

Build an abstraction:

- encapsulate the decision about what can change, e.g., what values can be added, in one place for facilitating experimentation with language design
- keep what is a fixed decision in the interpreter body

Other abstraction points are possible!

Encapsulating Design Decision Points

```
def interp(expr: Exp): Val = expr match {
  case Num(n)        => NumV(n)
  case Bool(b)       => BoolV(b)
  case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
  case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
  case Cond(c, t, e) => boolDec(interp(c)) match
      case true  => interp(t)
      case false => interp(e)
}
```

```
def addDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) + unboxNum(rhs))

def subDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) - unboxNum(rhs))

def boolDec(test: Val): Boolean = unboxBool(test)
```

1. Why apply the **`NumV`** constructor in **`addDec`** rather than in **`interp`**?

1. Why not pass **`interp`** of **`t`** and **`e`** in the Cond-branches to **`boolDec`**?

Encapsulating Design Decision Points

```
def interp(expr: Exp): Val = expr match {
  case Num(n)         => NumV(n)
  case Bool(b)        => BoolV(b)
  case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
  case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
  case Cond(c, t, e) => boolDec(interp(c)) match
      case true  => interp(t)
      case false => interp(e)
}
```

```
def addDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) + unboxNum(rhs))

def subDec(lhs: Val, rhs: Val): Val =
    NumV(unboxNum(lhs) - unboxNum(rhs))

def boolDec(test: Val): Boolean = unboxBool(test)
```

1. Why apply the **`NumV`** constructor in **`addDec`** rather than in **`interp`**?

1. Why not pass **`interp`** of **`t`** and **`e`** in the Cond-branches to **`boolDec`**?

1. What can be added may change, e.g., to add strings.
2. This would evaluate both of them, which is exactly what a conditional is supposed to avoid. A fixed decision – we encode it in the interpreter.

NAMES

Motivational Example 1

```
2 * (355 / 113) * 7
```

Can you explain this code?

```
val pi = 355 / 113
val radius = 7
val circumference = 2 * pi * radius
```

Names or identifiers serve as documentation
(Note: names are not variables – variables allow mutation)

Motivational Example 2

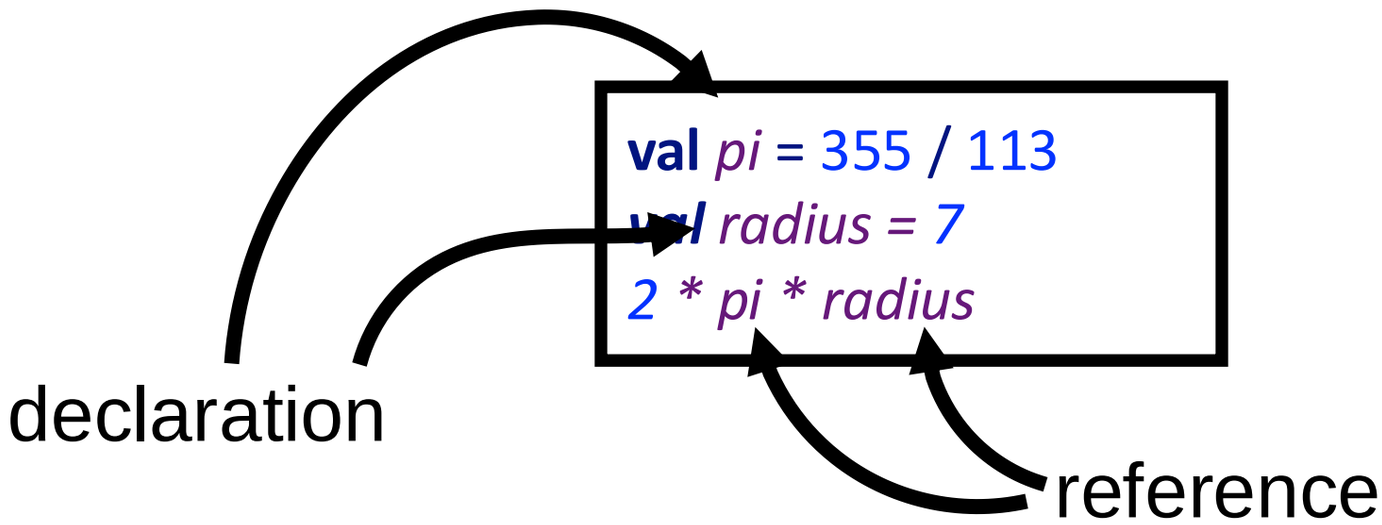
```
2 * (355 / 113) * 7 + 2 * (355 / 113) * 9
```

Can you explain this code?

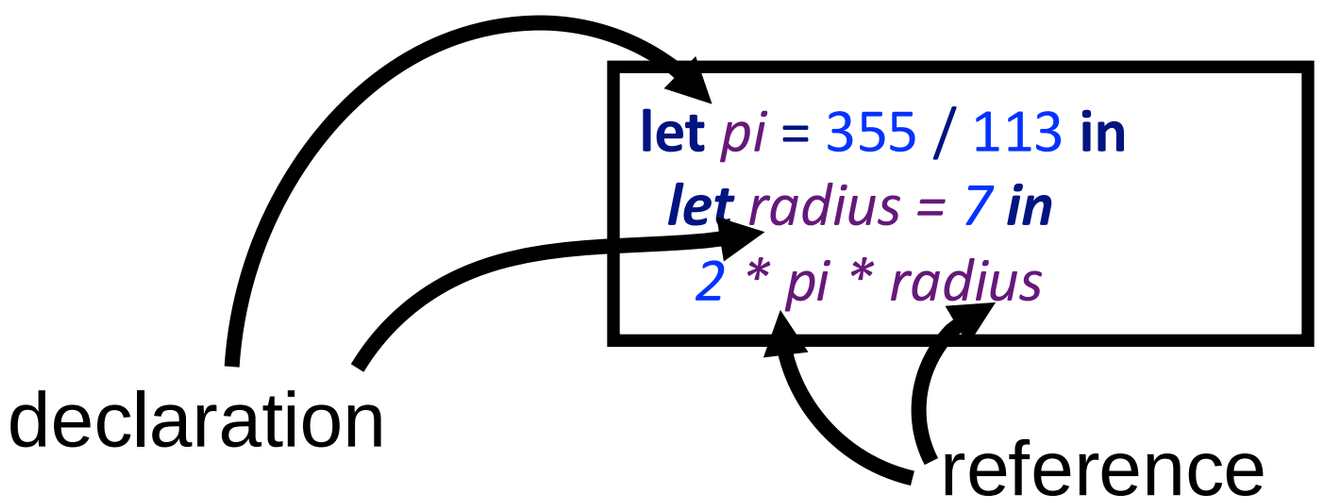
```
// computes circumference of circle with radius 7 and 9

val pi = 355 / 113
val circum1 = 2 * pi * 7
val circum2 = 2 * pi * 9
circum1 + circum2
```

Names help avoid redundancy/repetions

Declarations and References

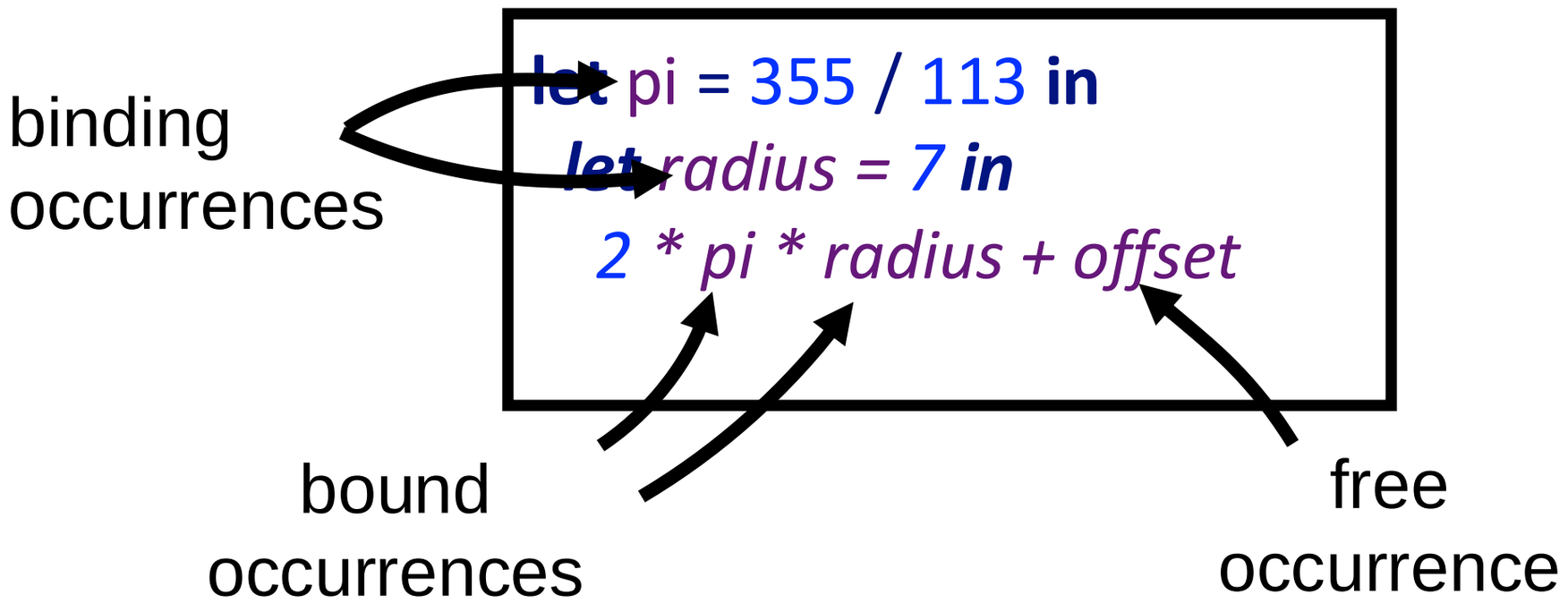


In Scala



Our lang

Binding, Bound, Free Occurrences



A name **`x`** is free in an expression **`e`** if **`x`** is not bound within **`e`**.

let *x* = 5 **in** *x* + *x*

x is free in (*x* + *x*)
x is bound in (**let** *x* = 5 **in** *x* + *x*)

Let’s Extend the Language with Let and Ids

Quiz: What steps are needed?

Concrete Syntax for the Language with Identifiers

```
Exp ::= Double                (Num)
      | (+ Exp Exp)           (Plus)
      | (* Exp Exp)           (Mult)
      | (/ Exp Exp)           (Div)
      | (let Symbol Exp Exp)  (Let)
      | Symbol                (Id)
```

Example program:

```
val x = 3 + 4
x * x
```

```
(let x (+ 3 4)
       (* x x))
```

Concrete s-expr Syntax

```
Exp ::= Double                (Num)
      | (+ Exp Exp)           (Plus)
      | (* Exp Exp)           (Mult)
      | (/ Exp Exp)           (Div)
      | (let Symbol Exp Exp) (Let)
      | Symbol                (Id)
```

Task: Translate the program in our s-expr language

```
val pi = 355 / 113
val radius = 7
2 * pi * radius
```

```
(let pi (/ 355 113)
  (let radius 7
    (* 2 (* pi radius))))
```

Abstract Syntax

```
Exp ::= Double                (Num)
      | (+ Exp Exp)           (Plus)
      | (* Exp Exp)           (Mult)
      | (/ Exp Exp)           (Div)
      | (let Symbol Exp Exp) (Let)
      | Symbol                (Id)
```

```scala
sealed trait Exp

case class Num(n: Double) extends Exp
case class Bool(b: Boolean) extends Expr

case class Add(lhs: Exp, rhs: Exp) extends Exp
case class Sub(lhs: Exp, rhs: Exp) extends Exp
case class Mult(lhs: Exp, rhs: Exp) extends Exp
case class Div(lhs: Exp, rhs: Exp) extends Exp
case class Cond(c: Exp, t: Exp, e: Exp) extends Exp
case class Let (name: String, bound: Exp, body: Exp) extends Exp
case class Id  (name: String) extends Exp
```

Semantics of Let

… by the concept of substitution

```
def interp(expr: Exp): Val = expr match {
  case Num(n)          => ...
  case Bool(b)         => ...
  case Add(lhs, rhs) => ...
  ...

  case Let(name, bound, body) =>
          interp(subst(name, bound, body))
  case Id(name) => ???
}
```

```
def subst(id: String, bound: Exp, body: Exp): Exp
```

Defining Substitution

Wanted: A definition of the process of *substitution*

Here is one:

> Definition (Substitution):
>
> To substitute identifier i in e with expression v, replace all identifier sub-expressions of e named i with v.

What do you think?

Defining Substitution

Wanted: A definition of the process of *substitution*

Here is one:

> Definition (Substitution):
>
> To substitute identifier i in e with expression v, replace all identifier sub-expressions of e named i with v.

Try it out with the following expressions:

```
let x = 5 in x + x
```

```
let x = 5 in
        x * (let x = 3 in x)
```

Our substitution converted a reasonable program into one that would be rejected by a parser, because it contains 5 where we expect an identifier. Obviously, the substitution algorithm is too naive.

Defining Substitution

Binding Instance: A binding instance of an identifier is the occurrence of the identifier that gives it its value. In our case, the <id> position of a ‘let’ is the only binding instance.

Scope: The scope of a binding instance, BI, is the program region, in which instances of the identifier refer to the value bound by BI.

Bound Instance: An identifier is bound if it is contained within the scope of a binding instance of its name.

Free Instance: An identifier not contained in the scope of any binding instance of its name is said to be free.

Defining Substitution

```
let x = 5 in
  x + (let x = 3 in
                x + x)
```

What‘s wrong here?

```
➔ 5 + (let x = 3 in
                5 + 5)
```

We do not respect scoping!

```
➔ 5 + (5 + 5)

➔ 15
```

Defining Substitution 2

Substitution: To substitute identifier i in e with expression v, replace all free instances of i in e with v.

- This definition implicitly uses a notion of scope
  - Substitute only in the scope of the binding instance of identifier

- An inner binding for the same name introduces a new scope.
  - The scope of the outer binding is shadowed or masked by the inner binding.

- Substituting the inner x is wrong.

Implementing Substitution

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  case Num(n)  => body
  case Bool(b) => body
  case Plus(l,r) => Plus(subst(id, bound, l), subst(id, bound, r))
  case Mult(l,r) => Mult(subst(id, bound, l), subst(id, bound, r))
  case Div(l,r)  => Div (subst(id, bound, l), subst(id, bound, r))

  case Let(y, boundE, bodyE) => ???
  case Id(y)  => ???
}
```

How to handle *Id(y)*?

Implementing Substitution

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  case Num(n)  => body
  case Bool(b) => body
  case Plus(l,r) => Plus(subst(id, bound, l), subst(id, bound, r))
  case Mult(l,r) => Mult(subst(id, bound, l), subst(id, bound, r))
  case Div(l,r)  => Div (subst(id, bound, l), subst(id, bound, r))

  case Let(y, boundE, bodyE) => ???

  case Id(y)  => if (y == id) bound else Id(y)
}
```

How to handle *Id(y)*?

If **`y`** in **`Id(y)`** is **`id`**, replace **`Id(y)`** with the expression bound to **`id`** (**`bound`**); otherwise return the expression **`Id(y)`** unchanged.

Implementing Substitution

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  case Num(n)  => body
  case Bool(b) => body
  case Plus(l,r) => Plus(subst(id, bound, l), subst(id, bound, r))
  case Mult(l,r) => Mult(subst(id, bound, l), subst(id, bound, r))
  case Div(l,r)  => Div (subst(id, bound, l), subst(id, bound, r))

  case Let(y, boundE, bodyE) => ???

  case Id(y)  => if (y == id) bound else Id(y)
}
```

How to handle *Let*?

Implementing Substitution

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  case Num(n)  => body
  case Bool(b) => body
  case Plus(l,r) => Plus(subst(id, bound, l), subst(id, bound, r))
  case Mult(l,r) => Mult(subst(id, bound, l), subst(id, bound, r))
  case Div(l,r)  => Div (subst(id, bound, l), subst(id, bound, r))

  case Let(y, boundE, bodyE) => ???

  case Id(y)  => if (y == id) bound else Id(y)
}
```

How to handle *Let*?

```
subst(x, (* 3 4), (let y (+ x 1) (/ x y)))
=                 (let y (+ (* 3 4) 1) (/ (* 3 4) y))

subst(y, (+ (* 3 4) 1), (/ (* 3 4) y))
=                       (/ (* 3 4) (+ (* 3 4) 1))
```

Implementing Substitution

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  ...
  case Let(y, boundE, bodyE) =>
       Let(y,
           subst(id, bound, boundE),
           subst(id, bound, bodyE))
```

What about this?

subst(x, (* 3 4), (let y (+ x 1) (/ x y)))
= (let y (+ (* 3 4) 1) (/ (* 3 4) y))

subst(y, (+ (* 3 4) 1), (/ (* 3 4) y))
= (/ (* 3 4) (+ (* 3 4) 1))

Substitution: Corner Case 1

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  ...
  case Let(y, boundE, bodyE) =>
       Let(y,
           subst(id, bound, boundE),
           subst(id, bound, bodyE))
```

Let's test with this?

```
(let x (* 3 4) (let x (+ x 1) (/ x x))
```

Our current implementation:

```
(let x (* 3 4) (let x (+ x 1) (/ x x))
  => (let x (+ (* 3 4) 1) (/ (* 3 4) (* 3 4)))
  => (/ (* 3 4) (* 3 4))
```

Remember: Variable shadowing!
The inner declaration of x <u>shadows</u> the outer declaration of x

Substitution: Take 2

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  ...

  case Let(y, boundE, bodyE) =>
      if (id = y)
       Let(y, subst(id, bound, boundE), bodyE)
      else
        Let(y,
            subst(id, bound, boundE),
            subst(id, bound, bodyE))
   ...
}
```

We don’t substitute in the body of inner let, if it binds the identifier of the outer let.

```
(let x (* 3 4) (let x (+ x 1) (/ x x))
  => (let x (+ (* 3 4) 1) (/ x x))
  => (/ (+ (* 3 4) 1) (+ (* 3 4) 1))
```

Substitution: Corner Case 2

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match {
  ...

  case Let(y, boundE, bodyE) =>
      if (id = y)
       Let(y, subst(id, bound, boundE), bodyE)
      else
        Let(y,
            subst(id, bound, boundE),
            subst(id, bound, bodyE))
  ...
}
```

Try this test now …

```
(let x y (let y 5 (/ x y))
```

Our current implementation:

```
(let x y (let y 5 (/ x y))
 =>    (let y 5 (/ y y))
 =>          (/ 5 5)
```

Problem: Variable capture.
The binding instance of y in the inner let accidentially <u>captures</u> the free variable y of the outer let.

Substitution: Take 3

```
def subst(id: String, bound: Exp, body: Exp): Exp = body match
  …
  case Let(letId, boundE, bodyE) =>
    if (id == letId) Let(y, subst(id, bound, boundE), bodyE)
    else
      if ( !freeVars(bound).contains(letId) )
          Let(letId, subst(id, bound, boundE), subst(id, bound, bodyE))
    else
      val z = freshName(freeVars(bound) ++ freeVars(bodyE))
      Let(z, subst(id, bound, boundE), subst(id, bound, subst(letId, Id(z), bodyE))
)
```

```
id = x,
bound = y,
body = (let y 5 (\ x y))

letId = y,
boundE = 5,
bodyE = (\ x y)
```

```
 (let x y (let y 5 (/ x y))
=>(let x y (let z 5 (/ x z))
=>(let z 5 (/ y z))
=>(/ y 5)
```

Capture-Avoiding Substitution

We have defined a capture-avoiding substitution function that implements sound variable replacement.

This is fundamental for programming languages and mathematics in general.

Alpha-Equivalence

Let(y, bound, body)

$\equiv_\alpha$

Let(z, bound, subst(y, Id(z), body)
if !freeVars(body).contains(z)

Alpha-equivalence:
Two terms are identical up to consistent renaming of variables.

- terms have the same structure
- references resolve to the same declarations
- but names used for references and declarations may differ

Interpreting Ids

```
def interp(expr: Exp): Val = expr match {
 case Num(n)          => NumV(n)
 case Bool(b)         => BoolV(b)
 case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
 case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
 case Cond(c, t, e) => boolDec(interp(c)) match
     case true  => interp(t)
     case false => interp(e)
 case Let(id, bound, body) => interp(subst(id, bound, body))
 case Id(x) => ???
}
```

subst(x, (* 3 4), (+ x y)) = (+ (* 3 4) y)

Semantics of Id

Any Id in the scope of a let-expr is replaced when the interpreter encounters that identifier's binding instance.

- subst replaces identifiers before the interpreter ever "sees" them.
- the interpreter can't replace a free name - it halts with an error

```
def interp(expr: Exp): Val = expr match {
  case Num(n)          => NumV(n)
  case Bool(b)         => BoolV(b)
  case Add(lhs, rhs) => addDec(interp(lhs), interp(rhs))
  case Sub(lhs, rhs) => subDec(interp(lhs), interp(rhs))
  case Cond(c, t, e) => boolDec(interp(c)) match
      case true  => interp(t)
      case false => interp(e)
  case Let(id, bound, body) => interp(subst(id, bound, body))
  case Id(x) => sys.error(s"Unbound variable $x")
}
```

Two Substitution Regimes

Eager substitution (static and dynamic reduction): avoids re-computing the same value several times.

1

```
  {let {x {+ 5 5}} {let {y {- x 3}} {+ y y}}}
= {let {x 10} {let {y {- x 3}} {+ y y}}}
= {let {y {- 10 3}} {+ y y}}
= {let {y 7} {+ y y}}
= {+ 7 7}
= 14
```

Lazy substitution (Static reduction): the expression may be evaluated multiple times.

2

```
  {let {x {+ 5 5}} {let {y {- x 3}} {+ y y}}}
= {let {y {- {+ 5 5} 3}} {+ y y}}
= {+ {- {+ 5 5} 3} {- {+ 5 5} 3}}
= {+ {- 10 3} {- {+ 5 5} 3}}
= {+ {- 10 3} {- 10 3}}
= {+ 7 {- 10 3}}
= {+ 7 7}
= 14
```

Two Substitution Regimes: Questions

1. Which one have we implemented?

```
Home work
```

1. Our example suggests that the eager regime generates an answer in fewer steps. Is this always true?

```
{let {x {+ 5 5}}
     {let {y 4} {+ y y}}}
```

2. Do the two regimes always produce the same result?

```
{let {x {+ z 4}}
     {let {y 4} {+ y y}}}
```

Notes on the Practicality of Substitution

Substitution is not practical for implementing programming languages. Any idea why?

Because our interpreter requires a full traversal of the body for each variable definition.

```
(let x (* 3 4) (let y (+ x 1) (let z 12 (/ y z)))

  =>  (let y (+ (* 3 4) 1) (let z 12 (/ y z))

  =>  (let z 12 (/ (+ (* 3 4) 1) z))

  =>  (/ (+ (* 3 4) 1) 12)
```